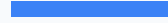


Mastering API Integrations in JavaScript

Fetch, asynchronous data flows, and architectural fundamentals

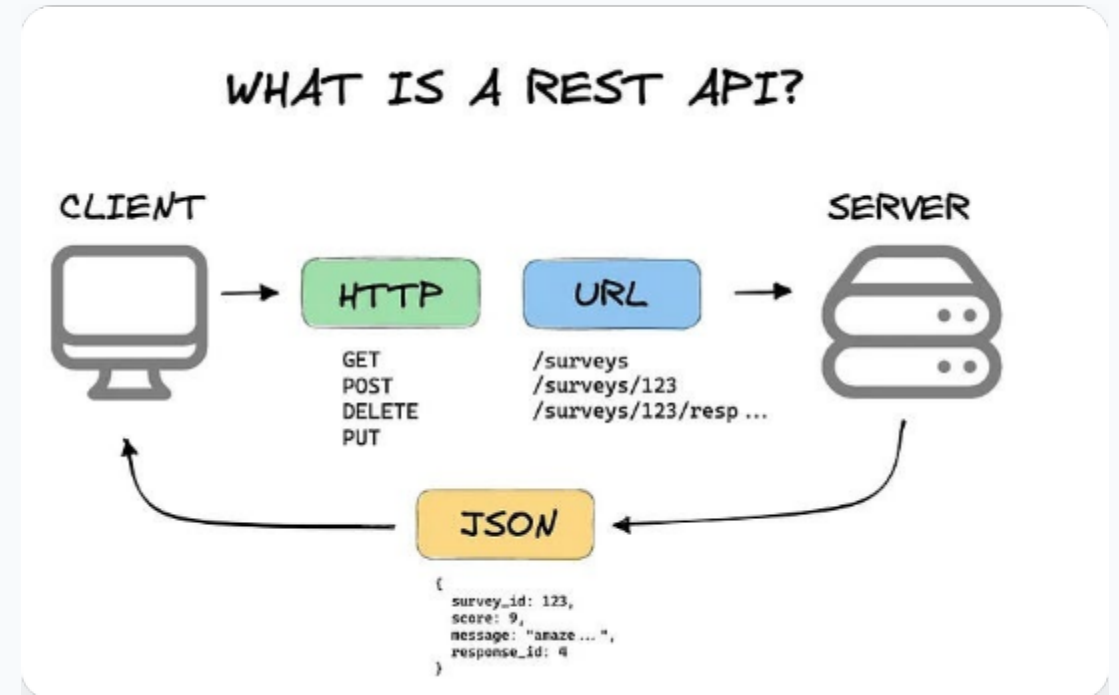


The Foundations

ARCHITECTURE

The Request Lifecycle

- **Client initiation:** Browser requests action.
- **Stateless transit:** HTTP carries the message.
- **JSON payload:** Universal data format.
- **Server response:** Status code and data.



VOCABULARY

Standard HTTP Methods

GET

Retrieve server data.

POST

Create new resources.

PUT

Update existing data.

DELETE

Remove specific records.

ANATOMY

The Endpoint URL

https://api.github.com/users/octocat?per_page=10

Protocol

HTTPS enforces encryption.

Host

The server domain.

Path

The specific resource.

Query

Optional filtering parameters.

RESPONSES

HTTP Status Codes

2xx

Success. The request was received and processed.

3xx

Redirection. Further action is needed.

4xx

Client Error. Bad syntax or unauthorized.

5xx

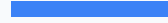
Server Error. The backend failed to fulfill.

PAYLOADS

The Universal Language

- **JSON:** JavaScript Object Notation.
- **String-based:** Easily transmitted over networks.
- **Strict syntax:** Double quotes required for keys.
- **Language agnostic:** Parsed natively by most backends.

```
// Raw JSON String representation  
{ "id": 101, "active": true, "roles": ["admin",  
"user"], "profile": { "theme": "dark" } }
```



The Fetch API

ASYNC FLOW

Awaiting the Response

1

fetch()

Initiate network call.

2

Headers Arrive

First Promise resolves.

3

.json()

Body parsing completes.

READING DATA

Anatomy of a GET

- **Defaults to GET:** No options object required.
- **Double await:** Wait for headers, then wait for body.
- **JSON parsing:** Native method converts buffer to object.

```
async function getUserData() {  
  try {  
    const res = await fetch('/api/user/1');  
    const data = await res.json();  
    console.log(data.name);  
  } catch (err) {  
    console.error(err);  
  }  
}
```

FILTERING

URLSearchParams

Never manually concatenate query strings (e.g., ?a=1&b=2).

Use the native browser API to handle special character encoding automatically.

```
const url = new URL('https://api.com/search');  
// Automatically handles ? and &  
const params = { q: 'javascript', limit: 10 };  
url.search = new URLSearchParams(params);  
const res = await fetch(url);
```

CRUCIAL BEHAVIOR

Fetch ignores 404s.

A Promise only rejects on complete network failure. Server error statuses (4xx, 5xx) still resolve successfully.

```
if (!res.ok) throw new Error('Fail');
```

WRITING DATA

Configuring a POST Request

```
const res = await fetch('/api/users', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify(newUser) // Must serialize object  
});
```

- Declare method explicitly.
- Set header for backend parsing.
- Stringify payload mandatory.

Identity via Headers

Secure APIs require credentials. The standard approach passes a token in the Authorization header.

```
const headers = { 'Authorization': `Bearer ${token}`, 'Content-Type': 'application/json' };
```

SECURITY POLICY

The CORS Block

Browsers block cross-origin requests by default. The server **MUST** explicitly allow your domain via the **Access-Control-Allow-Origin** header.



localhost:3000



BLOCKED



api.production.com

CONTROL FLOWS

Canceling Requests

- **Race conditions:** Prevent old slow responses from overwriting new data.
- **User navigation:** Cancel in-flight requests if the user leaves the page.
- **AbortController:** Native browser API to signal cancellation.

```
const controller = new AbortController();  
// Pass the signal to fetch  
fetch('/api/data', { signal: controller.signal });  
// Later: cancel the request  
controller.abort();
```

Fetch API vs Axios

Native Fetch

- Built into modern browsers.
- Manual JSON serialization.
- Manual HTTP error checks.
- Ideal for small projects.

Axios Library

- External NPM dependency.
- Automatic JSON parsing.
- Throws on 4xx/5xx codes.
- Standard for large apps.

REST vs WebSockets

REST (HTTP)

- **Stateless:** Connection closes immediately.
- **Pull:** Client must explicitly ask for data.
- **Overhead:** Headers sent every single time.
- Use for CRUD operations.

WebSockets

- **Stateful:** Persistent TCP connection.
- **Push:** Server sends data instantly.
- **Low latency:** Minimal framing.
- Use for chat, live feeds, gaming.